Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Bonnes pratiques

Organiser son code pour qu'il reste facile à changer

Le code de FraisDirect a grossi

- Une classe `Client` de 40 champs
- Un `CommandeService` qui envoie aussi les mails et les factures
- Une API qui renvoie tout, tout le temps
- Un package `service/` de 80 classes

Rien n'est faux. Mais chaque modification prend une journée.

Au programme

1. Découper ses objets : **SOLID** et la **composition**
2. Designer son **API REST**
3. Ranger ses **packages**

ACTE 1

Découper ses objets

Des classes petites, qui font une seule chose

EXERCICE

Petit exercice

1. Implémenter un objet `Afficheur` avec une méthode `afficher(String message)` qui fait `System.out.println(message)`
2. Ajouter une vérification : si le message contient un « a », on ne l'affiche pas
3. Dans un `main` : deux afficheurs, un en minuscules et un en majuscules
4. L'afficheur en majuscules vérifie aussi que le message fait 5 caractères
5. Un troisième affiche normalement et vérifie **uniquement** la taille de 5

Les principes SOLID

S	Single responsibility	Une classe, une seule raison de changer
O	Open/closed	Ajouter un comportement sans modifier l'existant
L	Liskov substitution	Une sous-classe peut remplacer sa mère
I	Interface segregation	Des petites interfaces plutôt qu'une grosse
D	Dependency inversion	Dépendre d'interfaces, pas d'implémentations

QUESTION

Une classe `ClientService` gère les clients, envoie les mails et génère les factures PDF. Quel principe viole-t-elle surtout ?

A. S : Single responsibility

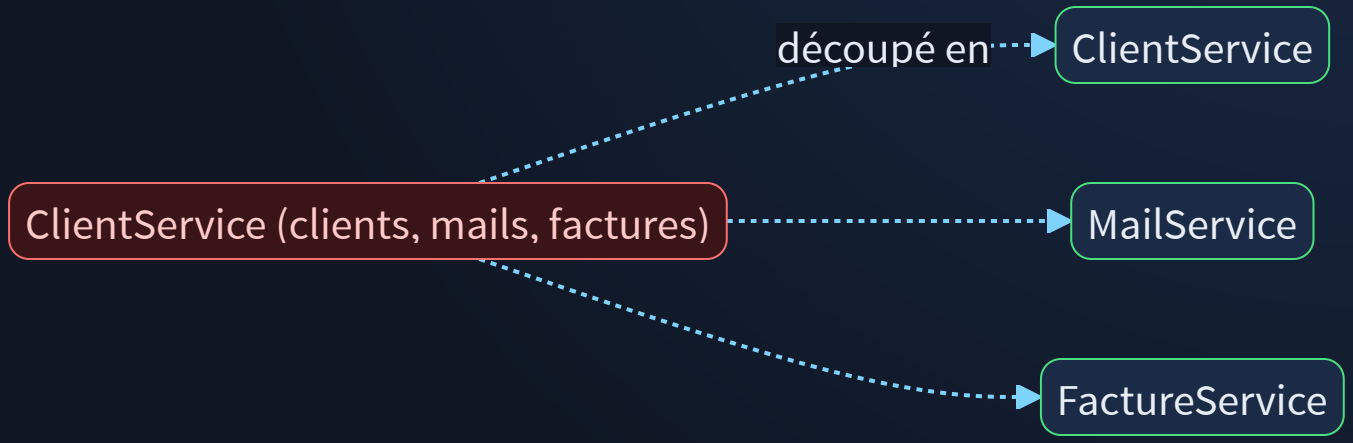
B. O : Open/closed

C. L : Liskov substitution

D. I : Interface segregation

E. D : Dependency inversion

S : découper un service



- Chaque classe a une seule raison de changer
- Chaque classe se teste seule, avec moins de mocks

S : les entités aussi

Les entités sont souvent la racine du problème

```
public class Client {
    private String nom;
    private String rue;
    private String ville;
    private String codePostal;
}
```

La rue, la ville et le code postal vont toujours ensemble…

Étape 1 · Un groupe logique

```
1  public class Client {
2      private String nom;
3      private Adresse adresse;
4  }
5
6  public class Adresse {
7      private String rue;
8      private String ville;
9      private String codePostal;
10 }
```

Étape 2 · Le besoin change

FraisDirect livre à une adresse et facture à une autre

```
public class Client {
    private String nom;
    private Adresse adresseLivraison;
    private Adresse adresseFacturation;
}
```

Une ligne de plus, au lieu de quatre champs copiés.

Étape 3 · Des rôles différents

Un client et un livreur ont tous les deux un compte

```java
public class Utilisateur {
    private String nom;
    private String email;
    private String motDePasse;
}

public class Client {
    private Utilisateur utilisateur;
    private Adresse adresseLivraison;
}

public class Livreur {
    private Utilisateur utilisateur;
    private String immatriculation;
}
```

Composition over inheritance

- L'héritage n'est pas la solution pour factoriser du code
- 90 % du temps, il est mal utilisé (source : moi)

Mauvais exemple

```java
abstract class Notification {
    abstract void envoyer(Commande commande);

    protected String formaterMontant(long centimes) {
        return String.format("%d,%02d €",
                centimes / 100, centimes % 100);
    }
}

class NotificationMail extends Notification {
    void envoyer(Commande commande) {
        String montant = formaterMontant(commande.montant());
        // … envoie le mail
    }
}
```

La mère n'appelle jamais `envoyer` : elle sert juste de boîte à outils

Bon exemple

```
abstract class Notification {
    final void notifier(Commande commande) {
        if (accepte(commande.client())) {
            envoyer(commande);
        }
    }

    abstract boolean accepte(Client client);
    abstract void envoyer(Commande commande);
}

class NotificationSms extends Notification {
    boolean accepte(Client client) {
        return client.accepteLesSms();
    }

    void envoyer(Commande commande) {
        // … envoie le SMS
    }
}
```

Retour à l'afficheur

Le format et la vérification deviennent des objets

```
class Afficheur {
    private final Function<String, String> format;
    private final Predicate<String> verification;

    Afficheur(Function<String, String> format,
              Predicate<String> verification) {
        this.format = format;
        this.verification = verification;
    }

    void afficher(String message) {
        if (verification.test(message)) {
            System.out.println(format.apply(message));
        }
    }
}
```

Retour à l'afficheur

```
1  Predicate<String> sansA = m -> !m.contains("a");
2  Predicate<String> cinqCaracteres = m -> m.length() == 5;
3
4  var minuscule = new Afficheur(String::toLowerCase, sansA);
5
6  var majuscule = new Afficheur(
7          String::toUpperCase,
8          sansA.and(cinqCaracteres));
9
10 var normal = new Afficheur(
11         Function.identity(), cinqCaracteres);
```

La consigne piège devient une ligne : on assemble autrement

À retenir

- Une classe, une seule raison de changer
- Des champs qui vont ensemble forment un objet
- Pour partager du code : une classe à part, injectée
- Une classe abstraite seulement si la mère appelle les méthodes des filles

ACTE 2

Designer son API REST

Ce que voit le front n'est pas ce qui est en base

QUESTION

Retourner directement l'entité JPA `Client` (avec son mot de passe) depuis un `@RestController`, bonne idée ?

Ce qui se passe

```java
@Entity
public class Client {
    @Id
    private Long id;
    private String email;
    private String motDePasse;
    @OneToMany(mappedBy = "client")
    private List<Commande> commandes;
}
```

- Le hash du mot de passe part dans le JSON
- Client → commandes → client → commandes… : sérialisation sans fin
- Renommer une colonne casse le front

La solution : un DTO

```java
public record ClientDto(Long id, String email) {}

@GetMapping("/clients/{id}")
public ClientDto trouver(@PathVariable Long id) {
    Client client = clientService.trouver(id);
    return new ClientDto(client.getId(), client.getEmail());
}
```

- On choisit exactement ce qui sort
- On maîtrise le schéma, et on peut le versionner

Découper les ressources

Un produit de FraisDirect

```java
public class Produit {
    private Long id;
    private String nom;
    private Categorie categorie;
    private List<Produit> similaires;
    private List<Avis> avis;
}
```

Mauvaise pratique

`GET /produits/12`

```
{
  "id": 12,
  "nom": "Comté 18 mois",
  "categorie": {
    "nom": "Fromages"
  },
  "similaires": [
    { "id": 13, "nom": "Comté 24 mois" }
  ],
  "avis": [
    { "note": 5, "texte": "Délicieux" },
    …
  ]
}
```

Bonne pratique

Les verbes HTTP

Je veux	Appel
Lire	`GET /commandes/12`
Chercher	`GET /commandes?client=42`
Créer	`POST /commandes`
Modifier	`PUT /commandes/12`
Supprimer	`DELETE /commandes/12`

Les verbes dans l'URL

- Pour une action métier : `POST /commandes/12/annuler`
- `/init` : le `POST` suffit
- `/search` : le `GET` suffit
- `/delete` : le verbe HTTP existe

QUESTION

Quel appel pour supprimer le produit 12 ?

A. `POST /produits/12/delete`

B. `DELETE /produits/12`

C. `GET /produits/delete?id=12`

D. `POST /produits/delete/12`

À retenir

- Jamais d'entité dans un contrôleur : un DTO
- Une ressource par URL, les listes à part
- Le verbe HTTP dit l'action, sauf pour une action métier

ACTE 3

Ranger ses packages

Trouver le code sans le chercher

Rangement 1 · Par couche

```
fraisdirect/
├─ config/
├─ controller/
├─ repository/
└─ service/
```

- ⊕ Simple pour démarrer
- ⊖ 80 classes dans `service/`
- ⊖ Toucher aux commandes = ouvrir 4 dossiers

Rangement 2 · Par domaine

```
fraisdirect/
├─ catalogue/
│  ├─ controller/
│  ├─ repository/
│  └─ service/
├─ commande/
│  ├─ controller/
│  ├─ repository/
│  └─ service/
└─ livraison/
   └─ …
```

- Les éléments sont regroupés par domaine fonctionnel
- On explore le projet par le métier

Rangement 3 · Domaine, entrées et sorties

```
commande/
├─ config/
├─ in/
│  ├─ controller/
│  └─ amqp/
├─ out/
│  ├─ repository/
│  └─ paiement/
└─ service/
```

- `in/` : ce qui appelle le domaine (HTTP, messages)
- `out/` : ce que le domaine appelle (base, API externes)
- ⊕ Les dépendances se voient
- ⊕ Les études d'impact sont plus faciles

À retenir

- Ranger par domaine, pas par couche
- Séparer ce qui entre et ce qui sort du domaine
- Le but : trouver le code et mesurer l'impact d'un changement